VoiceOps 2026: ZentroMart Voice Support Agent

Capstone Technical Report & System Architecture

Consulting & Analytics Club IIT Guwahati × Gnani.ai Capstone Project

Public GitHub Repository: https://github.com/saivardhankundala2005-sys/zentromart-voiceops.git

Reference Calendar Date: Fixed at `2026-10-20`

Core Stack: Google Gemini (gemini-3.5-flash-lite, gemini-embedding-001), Gnani.ai (gnani-prisma-v2.5 STT, timbre-v2.5 TTS), Python 3.14

---

1. Executive Summary & Problem Formulation

Customer support operations for modern Indian e-commerce (such as ZentroMart, spanning groceries and electronics) require rapid, accurate resolution across return eligibility, refund SLAs, delivery attempts, and damaged goods. While Indian consumers overwhelmingly prefer voice channels—frequently speaking in Indian English or code-switching with Hindi (*Hinglish*)—deploying unaugmented generative LLMs directly into customer service creates critical operational liabilities:

1. **Hallucination & Policy Contradictions**: Marketing materials and FAQs frequently generalize that refunds take *"5 to 7 business days"*, conflicting with binding internal policies where wallet refunds are instant within 2 hours.
2. **Calendar & Financial Math Inaccuracies**: Standard models frequently err when calculating return deadlines (calendar days) versus refund SLAs (business days, which exclude Sundays and warehouse holidays like 19 October 2026).
3. **Data Privacy & Unauthorized Disclosure**: Disclosing order details without authenticating caller identity violates data protection norms.
4. **Unauthorized Financial Commitments**: Unbounded agents frequently promise compensation (e.g. *"I have approved your refund of ■34,999"*) for high-value claims requiring manual human investigation.

To resolve these liabilities, we built **VoiceOps 2026: The ZentroMart Omnichannel Voice Support Agent**. Rather than fragmented chatbots, a single shared Python agent engine powers three production interfaces:

• **Interactive Gradio SaaS Web Application** (`app.py`) with Text Chat, Voice Assistant, Knowledge Base Browser, AGT-IDV Order Lookup, Evaluation Lab, and 1-Click Demo Scenarios.

• **Batch Audio CLI Runner** (`run.py` & `run.sh`) processing folders of customer voice clips into structured JSON metadata and synthesized reply WAV files.

• **Automated Benchmark Harness** (`scripts/evaluate.py`) evaluating baseline models, grounded RAG, ASR voice robustness, and multi-turn conversations against ground-truth unit tests.

---

2. What Was Built in Each Step

+---+

Customer Input Layer
(Web Chat / Microphone / Batch Audio CLI)

+----------------------+----------------------+

|

v

+---+

Gnani Prisma v2.5 STT Engine
(Indian English & Hinglish Support)

+----------------------+----------------------+

|

v

+---+

Speech Text Normalization
(Query Cleaning, Slang Map, Order ID Match)

+----------------------+----------------------+

|

v

+---+

	1. AGT-IDV Caller Identity Gate	
	(State machine: phone/pincode proof)	
	2. Hybrid RRF Knowledge Retriever	
	(Dense Gemini + BM25 + Precedence)	
	3. Deterministic Python Business Engine	
	(Holiday calendar, return SLA math)	
	4. Policy Escalation & Tools Bridge	
	(Damage > 10k, Ticket Creation)	

	5. Grounded LLM Response Generator	
	(Strict document adherence)	

+----------------------+----------------------+

|

v

+---+

Spoken Speech Normalization
(Dates -> "27 October 2026", IDs -> digits)

+----------------------+----------------------+

|

v

+---+

Gnani Timbre v2.5 TTS Engine
(Kaveri Voice Synthesis)

+---+

2.1 Step 1: Baseline Architecture & Prompting Setup

• **Baseline LLM**: Configured Google Gemini (`gemini-3.5-flash-lite`) operating purely on pre-trained parametric knowledge without document grounding or external tool access.

• **System Prompting**: Enforced Turn 1 AI identity introduction (*"Hello, I am ZentroMart's AI support assistant"*), polite refusals for unsupported questions, and structured JSON output mode.

• **Baseline Limitations**: Revealed fundamental LLM blind spots: unable to accurately state internal refund timelines, product warranty specifications, or festive sale return extensions.

2.2 Step 2: Grounded RAG with Hybrid RRF & Deterministic Business Calendar

• **Section-Aware Chunking**: Ingested 21 knowledge base markdown files (`kb/*.md`) split strictly on `## [SECTION-ID]` boundaries plus 70 product catalog SKUs (`products.csv`), yielding **154 structured chunks**.

• **Dense Vector Indexing**: Generated 3072-dimensional embeddings via `gemini-embedding-001`, persisted to `data_index/kb_index_cache.json` for instant startup.

• **Hybrid Reciprocal Rank Fusion (RRF)**: Merged dense cosine similarity with sparse BM25 scores, augmented with mathematical precedence boosting:

$$\text{Policy} (5) > \text{Agent Guide} (4) > \text{Help} (3) > \text{FAQ} (2) > \text{Marketing} (1)$$

• **Superseded Document Hard-Filtering**: Automatically filters documents with `status: superseded` (such as outdated electronics return policies), ensuring only active policies reach the context window.

• **Deterministic Python Business Calendar**: Replaced LLM arithmetic with exact Python date computations. Calculated Mon–Sat working days, skipping Sundays and official warehouse holidays from `holidays.csv` (Independence Day, Gandhi Jayanti, 19 October 2026, 09 November 2026), relative to the fixed reference date **`2026-10-20`**.

2.3 Step 3: Stateful Multi-Turn Agent with AGT-IDV & Policy Escalation

• **AGT-IDV Caller Identity Gate**: Designed a state machine (`idle` $\rightarrow$ `awaiting_proof` $\rightarrow$ `verified` / `locked_out`). Requires caller authentication (last 4 digits of registered phone or 6-digit delivery pincode) before disclosing order facts, with a strict 1-retry limit.

• **Tools Bridge & Secret Redaction**: Wrapped `lookup_order` and `create_ticket` with automated secret redaction (`[REDACTED_SECRET]`) in audit logs and enforced a minimum 20-character ticket summary constraint.

• **Deterministic Policy Escalation**: Automatically triggers escalation and ticket creation when:

• Consignment damage exceeds **■10,000** (e.g. 4K Smart TVs).

• Refund delays exceed **3 business days** past promised SLA.

• Repeat contacts reach ≥ 2 prior calls on the same order.

• Refuses unauthorized financial compensation while routing to the human Priority Resolution team.

2.4 Step 4: Omnichannel Speech Layer (Gnani STT & TTS Pipeline)

• **Gnani Prisma v2.5 STT**: Transcribes customer speech with dedicated Indian English (`en-IN`) and Hindi (`hi-IN`) acoustic models. Implemented audio validation and automatic splitting for clips exceeding 60 seconds.

• **Speech Text Normalization**: Pre-processes colloquial phrases and normalizes agent replies for speech: expands dates (*"27 October 2026"*), splits 8-digit order IDs into spaced digit groups (*"8978 2837"*), expands currency (*"rupees"*), and replaces abbreviations (*"cash on delivery"*, *"return to origin"*).

• **Gnani Timbre v2.5 TTS**: Synthesizes spoken output using the `Kaveri` voice, explicitly enforcing `model="timbre-v2.5"`.

• **Empty Audio Recovery**: Detects silent or unintelligible audio and returns a graceful voice prompt (*"Sorry, I didn't catch that. Could you please say it again?"*).

---

3. Empirical Evaluation: Step 1 vs Step 2 (and Step 3) Scores

Evaluation was conducted using our automated benchmark harness ([scripts/evaluate.py](file:///c:/Users/saiva/Downloads/TTT%20capstone%20project/scripts/evaluate.py)) across practice questions and multi-turn conversation sets.

3.1 Comprehensive Comparative Benchmark Table

Evaluation Phase	Total Samples	Must-Include Acc	Must-Not-Include Acc	Decision Acc	Tool Call Acc	Overall Pass Rate	Avg Latency
Step 1: Baseline LLM	5 Questions	**20.0%**	100.0%	100.0%	N/A	**20.0%**	**919.4 ms**
Step 2: Grounded RAG (Clean)	5 Questions	**80.0%**	100.0%	100.0%	N/A	**80.0%**	**2,098.3 ms**
Step 2: RAG (ASR Robustness)	5 Questions	**80.0%**	100.0%	100.0%	N/A	**80.0%**	**3,470.2 ms**
Step 3: Multi-turn Convs	22 Turns (5 Convs)	**100.0%**	100.0%	**95.45%**	**100.0%**	**95.45%**	**3,162.6 ms**

3.2 Deep-Dive Score Analysis & Grounding Efficacy

1. **Failure of the Baseline (20.0% Pass Rate)**:

The unaugmented baseline model scored only 20.0% on must-include factual precision. On `PQ0003` (*"How fast are refunds to the ZentroMart Wallet?"*), it lacked the 2-hour policy rule. On product catalog queries (`PQ0002`, `PQ0005`), it failed to confirm ZentroMart Assured coverage and 12-month brand warranty due to lack of catalog access.

2. **Precedence Defense & Neutralizing the FAQ Trap**:

In Step 2 Grounded RAG, hybrid precedence boosting correctly prioritized authoritative policy `REF-2` over `FAQ-1`. The agent correctly replied:

> *"Refunds to your ZentroMart Wallet are instant, within 2 hours of initiation."*

It achieved a **0.0% hard-negative hit rate**, completely rejecting the misleading 5-to-7 day FAQ timeline.

3. **Resilience to Speech-to-Text Noise**:

Testing on simulated speech transcripts (`asr_text`) featuring disfluencies (*"uh as a zentromart plus member will i be charged..."*, *"so sku ele-zero zero one to pe..."*) achieved an identical **80.0% pass rate**, demonstrating that query normalization and fuzzy matching insulate the agent against acoustic transcription noise.

4. **Stateful Conversation Precision (95.45% Pass Rate)**:

Across 22 turns in Step 3, the agent achieved **100.0% must-include accuracy** and **100.0% tool call accuracy**. Identity verification was enforced without exception on Turn 2, caller proof was verified on Turn 3, and high-value damage claims were escalated to Priority Resolution with ticket generation on Turn 4.

---

4. What Went Wrong & What We Would Fix

4.1 Engineering Challenges & What Went Wrong During Development

• **Gemini Free-Tier Rate Limits & Daily Quota**:

During multi-turn batch runs, `gemini-2.5-flash` hit strict free-tier quotas (5 requests/min and 20 requests/day), returning `429 RESOURCE_EXHAUSTED` with 15-hour retry delays.

Fix Applied: Enhanced `GeminiProvider` to parse `retryDelay` and dynamically switch to `gemini-3.5-flash-lite`, ensuring uninterrupted execution without crashes.

• **ASR Acoustic Timeouts on Pure Tones**:

Synthetic single-frequency sine wave test files caused Gnani STT gateway timeouts (504).

Fix Applied: Implemented an empty-transcript recovery handler in `process_voice_turn()` that catches missing/empty audio transcripts, prompts the user to repeat (*"Sorry, I didn't catch that..."*), and synthesizes audio without failing the pipeline.

• **Identity Proof Digit Extraction Collisions**:

Phrasings like *"Last 4 digits are 4533"* caused naive digit extractors to extract `44533` (including the word "4").

Fix Applied: Built `extract_verification_proof()` using regex to strip prefix phrases before isolating the 4-digit phone suffix or 6-digit delivery pincode.

• **Windows Console Unicode Character Encoding**:

Console output threw `charmap codec can't encode` errors when logging Hindi text or checkmark emojis on Windows powershell (`cp1252`).

Fix Applied: Configured `sys.stdout.reconfigure(encoding="utf-8", errors="replace")` across all entry points (`run.py`, `evaluate.py`, `presubmit_check.py`).

4.2 Architectural Roadmap: What We Would Fix & Extend

1. **Full-Duplex Streaming Audio**: The current architecture uses turn-by-turn REST calls (audio in $\rightarrow$ STT $\rightarrow$ LLM $\rightarrow$ TTS $\rightarrow$ audio out), resulting in a ~2.5s turnaround. We would migrate to Gnani's bi-directional gRPC streaming interface with streaming LLM token generation to achieve sub-800ms conversational interruption (*barge-in*) capabilities.

2. **Fine-Tuned Acoustic Noise Cancellation**: While Gnani Prisma v2.5 handles Indian English well, Indian street/market background noise degrades accuracy. We would add an upstream WebRTC VAD (Voice Activity Detection) filter and spectral noise gating.

3. **Coreference Resolution for Multiple Orders**: Currently, the agent tracks one primary order per session. We would introduce multi-order state tracking to handle cases where customers inquire about two separate orders in a single phone call.

---

5. Bonus Work Implemented

Beyond the mandatory core requirements, we engineered four enterprise-grade additions:

1. **Interactive 6-Tab Gradio SaaS Web Platform (`app.py`)**:

Built a complete dark-mode customer support operations center featuring:

• *Tab 1: Customer Support*: Text chat with live turn inspector displaying agent decisions, retrieved section IDs, and executed tool calls.

• *Tab 2: Voice Assistant*: Microphone recording and audio upload with real-time latency breakdowns (STT, Agent, TTS, Roundtrip).

• *Tab 3: Knowledge Base Browser*: Real-time search across all 154 indexed chunks with category filters and status indicators.

• *Tab 4: AGT-IDV Order Verification Demo*: Interactive demonstration of the caller authentication challenge, retry limiter, and derived order facts.

• *Tab 5: Evaluation Lab*: Live dashboard displaying comparative benchmark metrics.

• *Tab 6: System Diagnostics & 1-Click Demo Scenarios*: One-click launchers for tricky test cases (Precedence Trap, Festive Return Window, TV Damage Escalation).

2. **Automated Pre-Submission Compliance Verifier (`scripts/presubmit_check.py`)**:

Engineered a 6-stage automated validation script that checks for secret leaks, `.env.example` cleanliness, dataset isolation, CLI compliance, report artifacts, and pytest execution before submission.

3. **Resilient Multi-Model Failover (`src/voiceops/llm/provider.py`)**:

Implemented zero-downtime model failover across `gemini-3.5-flash-lite`, `gemini-3.5-flash`, and `gemini-2.5-flash` with exponential backoff.

4. **Comprehensive Test Suite (28 Unit Tests)**:

Authored 28 automated tests covering business calendar arithmetic, identity verification, tools bridge redaction, agent conversations, voice layer synthesis, and CLI runner execution.

---

6. Submission Deliverables & Repository Links

• **Public GitHub Repository**: https://github.com/saivardhankundala2005-sys/zentromart-voiceops.git

• **Batch Audio CLI Runner**: `python run.py --input-dir --output-dir `

• **Unix Shell Script**: `./run.sh --input-dir --output-dir `

• **Interactive Web App**: `python app.py` (running on `http://localhost:7860`)

• **PDF Report**: [report.pdf](file:///c:/Users/saiva/Downloads/TTT%20capstone%20project/report.pdf) (compiled via ReportLab)

• **Demo Video Script**: [demo_script.md](file:///c:/Users/saiva/Downloads/TTT%20capstone%20project/demo_script.md)

• **Sample Outputs**: [sample_outputs/](file:///c:/Users/saiva/Downloads/TTT%20capstone%20project/sample_outputs/) (includes genuine synthesized reply WAV `conv01_turn1_reply.wav`)

• **Submission ZIP Archive**: `voiceops_submission.zip`